Università
degli Studi
di Padova

Transform coding and JPEG

Marco Cagnazzo

Multimedia Communications

Outline

Outline

Recall: SQ, PQ, EC

- In **scalar quantization** each sample is quantized independently from the others and the resulting distortion may be large:

$$D = \sigma_X^2 2^{-2R} \qquad \text{UQ for uniform RV}$$

$$D = K_X \sigma_X^2 2^{-2R} \qquad \text{HR UQ for generic RV}$$

$$D = c_X \sigma_X^2 2^{-2R} \qquad \text{OQ for generic RV}$$

- If the L levels are equiprobable, the entropy of the quantization indexes is $\log_2 L$ and an arithmetic encoder has a rate $R \to H = \log_2 L$ for a sufficiently long symbol sequence
- To improve the performance one must exploit sample dependence:
  1. **Predictive quantization**: reduces the signal variance
  2. **Block coding**: exploits signal sparsity
  3. **Transform coding**: makes the signal sparse
- We have seen that PQ reduces the signal variance. Today we focus on block coding and transform coding.

Overview

1. **Block coding**: idea of jointly encoding M samples
2. **Huang-Schulteiss formula** answers the question: what is the optimal rate allocation for block coding?
3. **Coding gain** tells how much block coding improves with respect to sample-by-sample coding (only if data are sparse!)
4. **Orthogonal transforms** allow data sparsification
5. **Frequency transforms**: convenient special cases of orthogonal transforms
6. **JPEG** uses all these ideas to provide a simple and efficient image compression method

Outline

Block coding

Let us consider a block of M random variables $\boldsymbol{X} = [X_1, X_2, \dots X_M]^T$ (column vector)
The RV do not need to be independent neither i.d.
Let $\sigma_k^2 = \mathrm{Var}(X_k)$ and let c_k be the *shape factor* of X_k
The distortion of X_k with an optimal quantizer is:

$$D_k = c_k \sigma_k^2 2^{-2R_k}$$

where R_k is the bit rate used for X_k
Let us consider what happens if the M RV are quantized with a rate that is jointly decided such that the global distortion is minimized
The **global distortion** is

$$\mathcal{D} = \frac{1}{M}\mathrm{E}\left[\|\boldsymbol{X} - Q(\boldsymbol{X})\|^2\right] = \frac{1}{M}\mathrm{E}\left[(\boldsymbol{X} - Q(\boldsymbol{X}))^T(\boldsymbol{X} - Q(\boldsymbol{X}))\right] = \frac{1}{M}\mathrm{E}\left[\sum_{k=1}^{M}(X_k - Q(X_k))^2\right]$$

Block coding

Let us continue the calculation of $\mathcal{D}$:

$$\mathcal{D} = \frac{1}{M}\mathrm{E}\left[\sum_{k=1}^{M}(X_k - Q(X_k))^2\right] = \frac{1}{M}\sum_{k=1}^{M}\mathrm{E}\left[(X_k - Q(X_k))^2\right]$$
$$= \frac{1}{M}\sum_{k=1}^{M}\mathcal{D}_k = \frac{1}{M}\sum_{k=1}^{M}c_k\sigma_k^2 2^{-2R_k}$$

The problem of block coding, also known as **resource allocation problem** is to find the rate vector $\boldsymbol{R} = [R_1, \dots R_M]^T$ such that the global distortion is minimized **under a constraint on the total rate** $\sum_{k=1}^{M} R_k = R_{\mathrm{Tot}}$
This problem can be solved using *Lagrange multipliers*

Resource allocation: Problem formulation

- Minimize $\mathcal{D}$ under a rate constraint

$$\min \mathcal{D}(\boldsymbol{R}) = \frac{1}{M} \sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} \qquad \text{subject to} \sum_{k=0}^{M-1} R_k \leq R_{\text{Tot}}$$

Resource allocation: Problem formulation

- Minimize $\mathcal{D}$ under a rate constraint

$$\min \mathcal{D}(\boldsymbol{R}) = \frac{1}{M} \sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} \qquad \text{subject to} \sum_{k=0}^{M-1} R_k \leq R_{\text{Tot}}$$

- Lagrange method. Minimize:

$$J(\boldsymbol{R}, \lambda) = \frac{1}{M} \sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} + \lambda \left(\sum_{k=0}^{M-1} R_k - R_{\text{Tot}} \right)$$

Resource allocation: Problem formulation

- Minimize $\mathcal{D}$ under a rate constraint

$$\min \mathcal{D}(\boldsymbol{R}) = \frac{1}{M}\sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} \qquad \text{subject to} \sum_{k=0}^{M-1} R_k \leq R_{\text{Tot}}$$

- Lagrange method. Minimize:

$$J(\boldsymbol{R}, \lambda) = \frac{1}{M}\sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} + \lambda \left(\sum_{k=0}^{M-1} R_k - R_{\text{Tot}} \right)$$

- Solution (Huang-Schulteiss formula) :

$$R_k^* = \frac{R_{\text{Tot}}}{M} + \frac{1}{2}\log\left[\frac{c_k \sigma_k^2}{c_{\text{GM}}\sigma_{\text{GM}}^2}\right]$$

Proof of the Huang-Schulteiss formula

$$\frac{\partial J}{\partial R_k} = -\frac{2\ln 2}{M} c_k \sigma_k^2 2^{-2R_k} + \lambda \qquad \frac{\partial J}{\partial \lambda} = \sum_{k=0}^{M-1} R_k - R_{\mathrm{Tot}}$$

Setting $\nabla J(\boldsymbol{R}^*, \lambda^*) = 0$, we get:

$$-\frac{2\ln 2}{M} c_k \sigma_k^2 2^{-2R_k^*} + \lambda = 0 \qquad 2^{-2R_k^*} = \frac{M\lambda}{2\ln 2}\frac{1}{c_k\sigma_k^2}$$

$$-2R_k^* = \log_2\left(\frac{M\lambda}{2\ln 2}\right) + \log_2\left(\frac{1}{c_k\sigma_k^2}\right) \qquad R_k^* = \frac{1}{2}\log_2\left(\frac{2\ln 2}{M\lambda}\right) + \frac{1}{2}\log_2\left(c_k\sigma_k^2\right)$$

$$R_k^* = \lambda' + \frac{1}{2}\log_2\left(c_k\sigma_k^2\right)$$

where $\lambda' = \frac{1}{2}\log_2\left(\frac{2\ln 2}{M\lambda}\right)$. We find its value by imposing the constraint $\sum_{k=1}^{M} R_k^* = R_{\mathrm{Tot}}$, which correspond to $\frac{\partial J}{\partial \lambda} = 0$

Proof of the Huang-Schulteiss formula

$$R_{\mathrm{Tot}} = \sum_{k=1}^{M} R_k^* = M\lambda' + \frac{1}{2}\sum_{k=1}^{M} \log_2\left(c_k\sigma_k^2\right) \qquad \lambda' = \frac{R_{\mathrm{Tot}}}{M} - \frac{1}{2M}\sum_{k=1}^{M} \log_2\left(c_k\sigma_k^2\right)$$

$$\lambda' = \frac{R_{\mathrm{Tot}}}{M} - \frac{1}{2}\log_2 \prod_{k=1}^{M}\left(c_k\sigma_k^2\right)^{\frac{1}{M}} \qquad \lambda' = \frac{R_{\mathrm{Tot}}}{M} - \frac{1}{2}\log_2\left(c_{\mathrm{GM}}\sigma_{\mathrm{GM}}^2\right)$$

Now, replacing λ' in the expression of R_k^*, we find:

$$R_k^* = \lambda' + \frac{1}{2}\log_2\left(c_k\sigma_k^2\right) = \frac{R_{\mathrm{Tot}}}{M} - \frac{1}{2}\log_2\left(c_{\mathrm{GM}}\sigma_{\mathrm{GM}}^2\right) + \frac{1}{2}\log_2\left(c_k\sigma_k^2\right)$$
$$= \frac{R_{\mathrm{Tot}}}{M} + \frac{1}{2}\log_2\left(\frac{c_k\sigma_k^2}{c_{\mathrm{GM}}\sigma_{\mathrm{GM}}^2}\right) \square$$

Interpretation of HS formula

- A uniform resource allocation ($\bar{R} = R_{\mathrm{Tot}}/M$) is refined using variances
- Per-component distortion:

$$\mathcal{D}_k^* = c_k \sigma_k^2 2^{-2R_k^*} = c_k \sigma_k^2 2^{-2\bar{R}} \frac{c_{\mathrm{GM}} \sigma_{\mathrm{GM}}^2}{c_k \sigma_k^2} = c_{\mathrm{GM}} \sigma_{\mathrm{GM}}^2 2^{-2\bar{R}}$$

$$\mathcal{D}^* = \frac{1}{M} \sum_{k=0}^{M-1} \mathcal{D}_k^* = c_{\mathrm{GM}} \sigma_{\mathrm{GM}}^2 2^{-2\bar{R}}$$

- Gaussian case: $c_{\mathrm{GM}} = c_k = c_{\mathcal{N}}$ and thus

$$R_k^* = \bar{R} + \frac{1}{2} \log \left[\frac{\sigma_k^2}{\sigma_{GM}^2} \right] \qquad \mathcal{D}_k^* = c_{\mathcal{N}} \sigma_{\mathrm{GM}}^2 2^{-2\bar{R}}$$

$$\mathcal{D}^* = c_{\mathcal{N}} \sigma_{\mathrm{GM}}^2 2^{-2\bar{R}}$$

Reminder: Arithmetic and Geometric Means

For a set of M positive real numbers $\{z_1, z_2, \ldots, z_M\}$:

The **Arithmetic Mean (AM)** is:

$$z_{\mathrm{AM}} = \frac{1}{M}\sum_{k=1}^{M} z_k = \frac{z_1 + z_2 + \cdots + z_M}{M}$$

The **Geometric Mean (GM)** is:

$$z_{\mathrm{GM}} = \sqrt[M]{\prod_{k=1}^{M} z_k} = \sqrt[M]{z_1 \cdot z_2 \cdot \ldots \cdot z_M}$$

- $z_{\mathrm{GM}} \leq z_{\mathrm{AM}}$, can be proved through Jensen's inequality, stating that for a strictly concave function f we have: $f\left(\sum_{k=1}^{M} \frac{1}{M} z_k\right) \geq \sum_{k=1}^{M} \frac{1}{M} f(z_k)$
- Equality holds if and only if all z_k are equal
- The geometric mean becomes smaller as the values become more different from each other

Case of i.d. random variables

Coding of multimedia signals: the samples are reasonably identically distributed, even though they are not independent
In this case,

$$\forall k \in \{1, \dots, M\}\ \sigma_k^2 = \sigma_X^2 \qquad \sigma_{\mathrm{GM}}^2 = \sigma_X^2$$

$$c_k = c_X \qquad c_{\mathrm{GM}} = c_X$$

$$R_k^* = \bar{R} + \frac{1}{2}\log_2 \frac{c_k \sigma_k^2}{c_{\mathrm{GM}} \sigma_{\mathrm{GM}}^2} = \bar{R}$$

$$\mathcal{D}_k^* = c_{\mathrm{GM}} \sigma_{\mathrm{GM}}^2 2^{-2\bar{R}} = c_X \sigma_X^2 2^{-2\bar{R}} \qquad \mathcal{D} = c_{\mathrm{GM}} \sigma_{\mathrm{GM}}^2 2^{-2\bar{R}} = c_X \sigma_X^2 2^{-2\bar{R}}$$

Samples are i.d., the signal is not sparse (all the samples are equally important). Thus, block coding does not really brings in any improvement. But the distortion might be reduced if we manage to reduce σ_{GM}^2

Outline

Signal sparsification

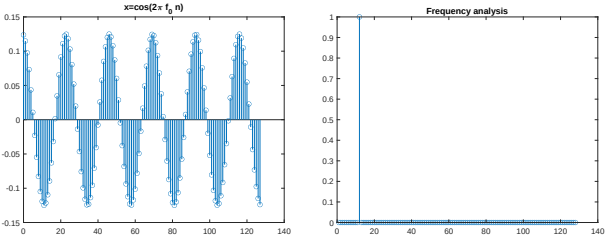
- Linear transform is a base changement aiming to make the signal **sparse**
- A sparse signal has a few large samples and many small samples
- Quantization over a sparse signal can be performed efficiently: use a high-resolution (ie., high bit-rate) quantizer on the “important” coefficient and a low-resolution quantizer on the irrelevant ones
- Practical versions of the H.S. formula can be used to decide the coding rate of the components of a sparse signal
- The sparsification goal is to transform a block of i.d. RVs into another block where the variances are as diverse as possible (to minimize σ^2_{GM})

Signal sparsification

We look for an operator T such that:

- Is reversible: $\boldsymbol{Y} = T(\boldsymbol{X}) \Leftrightarrow \boldsymbol{X} = T^{-1}(\boldsymbol{Y})$
- Input data is as a vector in $\mathbb{R}^M$
  - Example: M samples of recorded voice
  - Example: $M = M_1 \times M_2$ pixels from a rectangular block of a gray-level image
- Input data is **not sparse**
  - This means that any sample is potentially meaningful
- Output data $\boldsymbol{Y} = T(\boldsymbol{X})$ is **sparse** and the quantization error on $\boldsymbol{Y}$ is the same as on $\boldsymbol{X}$
  - A few non-negligible samples and many negligible (near zero) samples
  - If the quantization error is not the same, we risk that after reversing the operator, the error increases

Frequency analysis as sparsification



Left. A vector $\boldsymbol{x}$ of M samples from a sound signal (pure tone). The signal is non-sparse as all the samples are equally important

Right. The "transformed" vector $\boldsymbol{y} = A\boldsymbol{x}$, where A is an orthogonal matrix allowing to perform a frequency analysis. The resulting signal is **sparse**: one non-negligible sample and $N - 1$ negligible samples.

In general a sparse signal is made up of "a few" non-negligible samples and "many" negligible samples

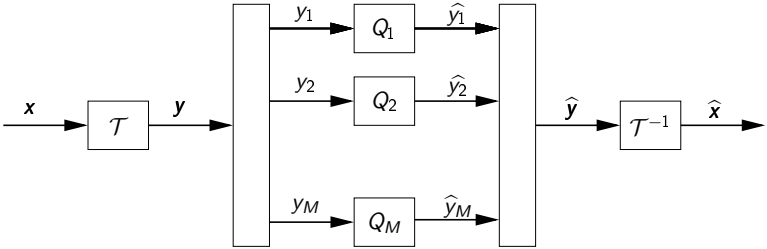
Linear Transforms as sparsifying operators

Let us consider $\boldsymbol{Y} = \mathcal{T}\boldsymbol{X}$, where $\mathcal{T}$ is an invertible matrix

- We can always find back $\boldsymbol{X}$ from $\boldsymbol{Y}$ using the inverse matrix $\mathcal{T}^{-1}$
- It can be interpreted as basis change i.e. a coordinate change
  - As such, the basis is a set of signals which are used to reconstruct the intended signal
  - Therefore the basis should be relevant to the nature of the signals
  - Sinusoidal signals appear to be a good choice for sound
  - It is also a reasonable choice for images in particular considering the contrast sensitivity dependence on frequency in HVS
- If the matrix is **orthogonal**, the quantization error is the same on $\boldsymbol{X}$ and $\boldsymbol{Y}$ (see later)
- We should look for the matrix $\mathcal{T}$ that minimizes the GM of the variances of $\boldsymbol{Y} = \mathcal{T}\boldsymbol{X}$

Linear Transforms

Paradigm of transform coding:



From $\boldsymbol{x}$ to $\boldsymbol{y} = \mathcal{T}\boldsymbol{x}$: we want an "easier" vector for coding, i.e. with a few large coefficients and many small ones

Orthogonal Transforms

An orthogonal transform (OT) means that the matrix $\mathcal{T}$ is orthogonal and thus $\mathcal{T}^{-1} = \mathcal{T}^T$, $\boldsymbol{Y} = \mathcal{T}\boldsymbol{X}$ and $\boldsymbol{X} = \mathcal{T}^T\boldsymbol{Y}$

1. For orthogonal transforms, the inversion is immediate
2. OT are isometries, i.e. they keep the $\mathcal{L}^2$ norm: for any $\boldsymbol{X}$,

$$\|\mathcal{T}\boldsymbol{X}\|^2 = (\mathcal{T}\boldsymbol{X})^T(\mathcal{T}\boldsymbol{X}) = \left(\boldsymbol{X}^T\mathcal{T}^T\right)(\mathcal{T}\boldsymbol{X}) = \boldsymbol{X}^T\left(\mathcal{T}^T\mathcal{T}\right)\boldsymbol{X} = \boldsymbol{X}^T\boldsymbol{X} = \|\boldsymbol{X}\|^2$$

The isometry property is very important because it insures that the distortion on $\boldsymbol{Y}$ is the same as the distortion on $\boldsymbol{X}$:

$$\mathcal{D}_Y = \frac{1}{M}\mathrm{E}\left[\|\boldsymbol{Y} - \widehat{\boldsymbol{Y}}\|^2\right] = \frac{1}{M}\mathrm{E}\left[\|\mathcal{T}\left(\boldsymbol{X} - \widehat{\boldsymbol{X}}\right)\|^2\right] = \frac{1}{M}\mathrm{E}\left[\|\left(\boldsymbol{X} - \widehat{\boldsymbol{X}}\right)\|^2\right] = \mathcal{D}_X$$

It is very important to keep the same distortion in the transform and in the original domain: only in this way the distortion minimization over $\boldsymbol{Y}$ has the same effect over $\boldsymbol{X}$

Orthogonal Transforms and Coding Gain

- Let $\boldsymbol{X}$ be a random vector of M components (sound, image...)
- Hypothesis: $\boldsymbol{X}$ components are identically distributed (i.d.) Gaussian RVs: $\forall k, X_k \sim \mathcal{N}(0, \sigma_X^2)$
  - Note that $\boldsymbol{X}$ components are not assumed to be independent
- In this case resource allocation is trivial since all the components have the same variance and shape factor. For historical reasons, the distortion in this case (non-transformed, i.d. RVs) is referred to as $\mathcal{D}_{\mathrm{PCM}}$, so we get

$$\mathcal{D}_{\mathrm{PCM}} = c_{\mathcal{N}} \sigma_X^2 2^{-2\bar{R}}$$

- Let us now consider a generic OT, such that $\boldsymbol{Y} = \mathcal{T}X$
- **Question: how much can we reduce the quantization distortion by applying the transform coding paradigm?**

Orthogonal Transforms and Coding Gain

- Remember that the transform coding paradigm consists in computing $\boldsymbol{Y} = \mathcal{T}\boldsymbol{X}$, apply quantization to $\boldsymbol{Y}$, and obtain $\widehat{\boldsymbol{Y}} = Q(\boldsymbol{Y})$
- The decoded version of $\boldsymbol{X}$ is $\widehat{\boldsymbol{X}} = \mathcal{T}^{-1}\widehat{\boldsymbol{Y}}$
- Let us compute the distortion of $\boldsymbol{X}$ in the case of transform coding, $\mathcal{D}_T$
- Remember that, since we use **orthogonal transforms**, the distortion can be computed directly on $\boldsymbol{Y}$:

$$\mathcal{D}_T = \frac{1}{M}\mathrm{E}\left\|\boldsymbol{X} - \widehat{\boldsymbol{X}}\right\|^2 = \frac{1}{M}\mathrm{E}\left\|\mathcal{T}^{-1}\boldsymbol{Y} - \mathcal{T}^{-1}\widehat{\boldsymbol{Y}}\right\|^2 = \frac{1}{M}\mathrm{E}\left\|\boldsymbol{Y} - \widehat{\boldsymbol{Y}}\right\|^2 = \mathcal{D}_Y$$

Orthogonal Transforms and Coding Gain

Assuming that resource allocation is done with optimal (H.S.) method, we get:

$$\mathcal{D}_Y = c_{\mathrm{GM},Y}\sigma_{\mathrm{GM},Y}2^{-2\bar{R}}$$

If $\boldsymbol{X}$ is Gaussian, also $\boldsymbol{Y}$ is Gaussian and thus $c_{\mathrm{GM},Y} = c_{\mathrm{GM},X} = c_{\mathcal{N}}$
Moreover, for any OT $\mathcal{T}$, we have

$$\sigma^2_{\mathrm{AM},Y} = \frac{1}{M}\sum_{k=1}^{M}\mathrm{E}\left[Y_k^2\right] = \frac{1}{M}\mathrm{E}\left[\sum_{k=1}^{M}Y_k^2\right] = \frac{1}{M}\mathrm{E}\left\|\boldsymbol{Y}\right\|^2 = \frac{1}{M}\mathrm{E}\left\|\boldsymbol{X}\right\|^2$$
$$= \frac{1}{M}\mathrm{E}\left[\sum_{k=1}^{M}X_k^2\right] = \frac{1}{M}\sum_{k=1}^{M}\mathrm{E}\left[X_k^2\right] = \sigma^2_{\mathrm{AM},X} = \sigma^2_X$$

Thus any OT $\mathcal{T}$ does not change the variance AM, but changes the variance GM

Orthogonal Transforms and Coding Gain

- The ***Coding Gain*** of the transform $\mathcal{T}$ is the ratio between the distortion on the original data and the transform coding distortion:

$$G_{\mathcal{T}} = \frac{\mathcal{D}_{\mathrm{PCM}}}{\mathcal{D}_{\mathcal{T}}} = \frac{c_{\mathcal{N}} \sigma_X^2 2^{-2\bar{R}}}{c_{\mathcal{N}} \sigma_{\mathrm{GM},Y} 2^{-2\bar{R}}} = \frac{\sigma^2_{\mathrm{AM},Y}}{\sigma^2_{\mathrm{GM},Y}}$$

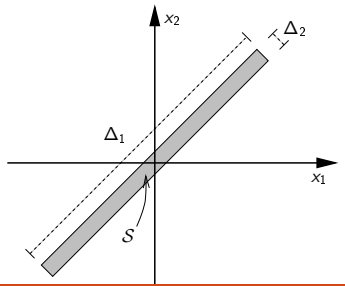
- The ratio between the AM and the GM of the variances of $\boldsymbol{Y}$ is the so-called **coding gain** of the transform
- Remember that for any OT $\mathcal{T}$, $\sigma^2_{\mathrm{AM},Y}$ is always the same: thus we look for a transform that minimizes $\sigma^2_{\mathrm{GM},Y}$
- This supports the idea that the transform must sparsify the signal, i.e. make the variances as diverse as possible

Transform coding: Toy Example

Let us consider now a “toy example” of transform coding. The goal is to show that if we have a strongly correlated signal, when we quantize each sample independently from the others, we cannot take advantage from this correlation.
On the contrary, after a linear transform the samples are “less correlated” (in this special example, they happen to become independent) and the couple is “sparse”, ie., one component is important, the other much less.
We will compute the RD curve for the two cases and show the advantage of transform coding.

Transform coding: Toy Example

Couple of highly correlated RV's: once we know the value of X_1, X_2 is constrained into a small interval. Nevertheless, each variable has the same marginal distribution. This could be a relatively good model for a couple of neighboring pixels in an image



$$f_{X_1,X_2}(x_1,x_2) = \begin{cases} \frac{1}{\Delta_1 \Delta_2} & \text{if } (x_1,x_2) \in \mathcal{S} \\ 0 & \text{if } (x_1,x_2) \notin \mathcal{S} \end{cases}$$

$$\Delta_1 \gg \Delta_2$$

$$X_1 \sim X_2 \sim \mathcal{U}\left[-\frac{\Delta_1}{2\sqrt{2}}, \frac{\Delta_1}{2\sqrt{2}}\right]$$

Transform coding: Toy Example

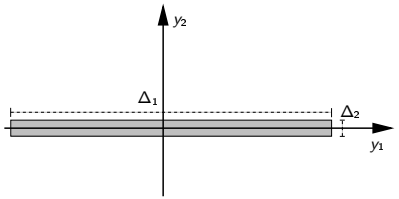
- We perform uniform quantization, since this is the best option in the case of uniform variables
- The rate-distortion relationship for X_i is then $D_i(R_i) = \sigma_i^2 2^{-2R_i}$
- The total distortion is $D = \sum_i D_i$
- $\sigma_1^2 = \sigma_2^2 = \sigma^2 = \left(\frac{\Delta_1}{\sqrt{2}}\right)^2 \frac{1}{12} = \frac{\Delta_1^2}{24}$

bits	R	D_1	D_2	D
0	0	σ^2	σ^2	$2\sigma^2$
1	0.5	$\sigma^2/4$	σ^2	$\frac{5}{4}\sigma^2$
1	0.5	σ^2	$\sigma^2/4$	$\frac{5}{4}\sigma^2$
2	1	$\sigma^2/4$	$\sigma^2/4$	$\frac{1}{2}\sigma^2$
2	1	$\sigma^2/16$	σ^2	$\frac{17}{16}\sigma^2$
3	1.5	$\sigma^2/16$	$\sigma^2/4$	$\frac{5}{16}\sigma^2$
4	2	$\sigma^2/16$	$\sigma^2/16$	$\frac{1}{8}\sigma^2$

Transform coding: Toy Example

We apply the transform: $\begin{bmatrix} Y_1 \\ Y_2 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$

After the transform the RV are independent



$$f_{Y_1,Y_2}(y_1,y_2) = \begin{cases} \frac{1}{\Delta_1 \Delta_2} & \text{if } (y_1,y_2) \in \mathcal{S} \\ 0 & \text{if } (y_1,y_2) \notin \mathcal{S} \end{cases}$$

$$Y_1 \sim \mathcal{U}\left[-\frac{\Delta_1}{2}, \frac{\Delta_1}{2}\right]$$

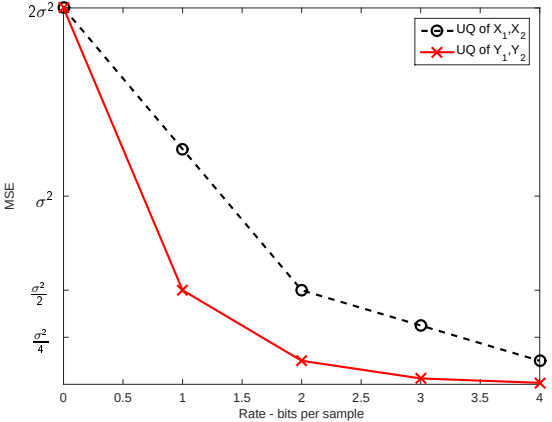
$$Y_2 \sim \mathcal{U}\left[-\frac{\Delta_2}{2}, \frac{\Delta_2}{2}\right]$$

Transform coding: Toy Example

- Quantization of Y_1 and Y_2: $D(R)$ curve
- $\sigma_1^2 = \frac{\Delta_1^2}{12} = 2\sigma^2$
- $\sigma_2^2 = \frac{\Delta_2^2}{12} \ll \sigma^2$

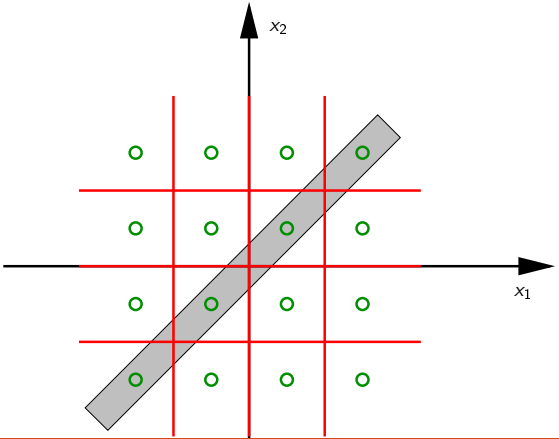
bits	R	D_1	D_2	D
0	0	$2\sigma^2$	$\ll \sigma^2$	$2\sigma^2$
1	0.5	$\sigma^2/2$	$\ll \sigma^2$	$\frac{1}{2}\sigma^2$
2	1	$\sigma^2/8$	$\ll \sigma^2$	$\frac{1}{8}\sigma^2$
3	1.5	$\sigma^2/32$	$\ll \sigma^2$	$\frac{1}{32}\sigma^2$

Transform coding: Toy Example



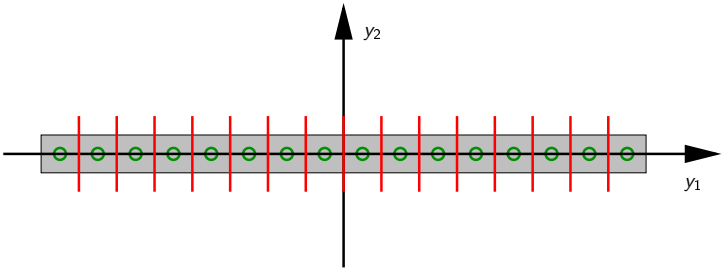
Transform coding: Toy Example

Scalar quantization in the original domain



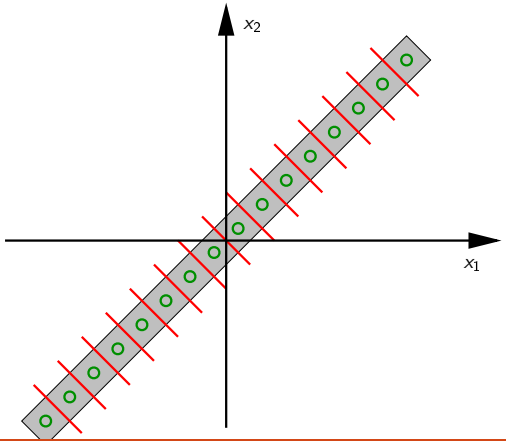
Transform coding: Toy Example

Scalar quantization in the transform domain



Transform coding: Toy Example

Vector quantization in the original domain



Practical Algorithms for Resource allocations

Huang-Schulteiss' formula

- may provide negative values
- may provide non-integer values

Practical (sub-optimal) algorithms

- Modified HS (Huang-Schulteiss) Algorithm
- *Greedy* Algorithm

Modified HS Algorithm

1. We compute R_k^* with Huang-Schulteiss ;
2. If some R_k^* are negative, we remove the variances of the concerned components and we repeat the computation. The removed variables are not coded (i.e, “coded with zero bits”)
3. This step is repeated as long as there are negative allocation values.
4. The results are floored
5. The eventual residual rate is allocated to coefficients with the largest error

Modified H.S.: Example

Initial Data

- Gaussian vector: $\forall k, c_k = c_{\mathcal{N}} \Rightarrow c_{\mathrm{GM}} = c_{\mathcal{N}}$
- Number of components: $M = 4$
- Variances: $\sigma_1^2 = 1000$, $\sigma_2^2 = 100$, $\sigma_3^2 = 50$, $\sigma_4^2 = 1$
- Geometric mean: $(1000 \cdot 100 \cdot 50 \cdot 1)^{1/4} \approx 47.29$
- Total rate: $R_{\mathrm{Tot}} = 10$ bits
- Huang-Schultheiss formula:

$$R(k) = \frac{R_{Tot}}{M} + 0.5 \log_2 \left(\frac{c_k \sigma_k^2}{c_{\mathrm{GM}} \sigma_{\mathrm{GM}}^2} \right) = 2.5 + 0.5 \log_2 \left(\frac{\sigma_k^2}{47.29} \right)$$

Modified H.S.: Example

- Initial R values:

$$R(1) \approx 4.70 \qquad R(2) \approx 3.04$$
$$R(3) \approx 2.54 \qquad R(4) \approx -0.28$$

- The rates are not all positive: we remove the variance corresponding to the minimum (most negative) rate, set its rate to zero, and continue with $M - 1$ variances
- $\mathrm{GM}(1000, 100, 50) \approx 171.00$
- H.S. formula on active variances:

$$R(k) = \frac{10}{3} + 0.5 \log_2 \frac{\sigma_k^2}{171.00}$$

Modified H.S.: Example

R values:

$$R(1) \approx 4.61$$
$$R(2) \approx 2.95$$
$$R(3) \approx 2.45$$
$$R(4) = 0$$

All rates are ≥ 0, rounding down:

$$R(1) = 4$$
$$R(2) = 2$$
$$R(3) = 2$$
$$R(4) = 0$$

- Total allocated bits: 4 + 4 + 2 + 0 = 8
- Residual bits: 10 - 8 = 2
- Largest fractional parts: R(2) and R(1)

Modified H.S.: Example

- Allocate residual bits to components with largest fractional parts:

$$R(1) = 5$$
$$R(2) = 3$$
$$R(3) = 2$$
$$R(4) = 0$$

Distortion Calculation:

$$D_1 = 1000 \cdot 2^{-10} \approx 0.98$$
$$D_2 = 100 \cdot 2^{-6} \approx 1.56$$
$$D_3 = 50 \cdot 2^{-4} \approx 3.13$$
$$D_4 = 1 \cdot 2^{0} = 1$$

$$D_{total} = D_1 + D_2 + D_3 + D_4 \approx 6.67$$

Greedy algorithm

The same allocation as the modified H.S. can be obtained with a greedy algorithm, which can be faster if R_{Tot} is small.
The algorithms is as follows

1. Initialization
   - $R_k = 0 \quad \forall k \in \{0, 1, \dots M-1\}$.
   - $D_k = \quad \sigma_k^2 \forall k \in \{0, 1, \dots M-1\}$.
2. While $\sum_k R_k \leq R_{\mathrm{Tot}}$
   - $\ell = \arg\max_k D_k$
   - $R_\ell \leftarrow R_\ell + 1$
   - $D_\ell \leftarrow D_\ell / 4$

Let us see this algorithm at work

Greedy Algorithm: Example

- Number of components: M = 4
- Variances:
  - $\sigma_1^2 = 100$
  - $\sigma_2^2 = 50$
  - $\sigma_3^2 = 10$
  - $\sigma_4^2 = 1$
- Total rate: $R_{Tot} = 10$ bits

Greedy Algorithm - Iterations 1-5

Iter	ℓ	R_1	R_2	R_3	R_4	D_1	D_2	D_3	D_4
0	-	0	0	0	0	1000.00	100.00	50.00	1.00
1	1	1	0	0	0	250.00	100.00	50.00	1.00
2	1	2	0	0	0	62.50	100.00	50.00	1.00
3	2	2	1	0	0	62.50	25.00	50.00	1.00
4	1	3	1	0	0	15.63	25.00	50.00	1.00
5	3	3	1	1	0	15.63	25.00	12.50	1.00

Greedy Algorithm - Iterations 6-10

Iter	ℓ	R_1	R_2	R_3	R_4	D_1	D_2	D_3	D_4
6	2	3	2	1	0	15.63	6.25	12.50	1.00
7	1	4	2	1	0	3.91	6.25	12.50	1.00
8	3	4	2	2	0	3.91	6.25	3.13	1.00
9	2	4	3	2	0	3.91	1.56	3.13	1.00
10	1	5	3	2	0	0.98	1.56	3.13	1.00

It is the same rate vector as with the modified H.S. algorithm

Outline

Orthogonal Transforms for Compression

- For effective compression, we need an orthogonal transform that maximizes the coding gain
- A linear transform can be represented as a matrix product:

$$\mathrm{y} = \mathcal{T}\mathrm{x}$$

where $\mathcal{T}$ is the transformation matrix

- For an orthogonal transform: $\mathcal{T}\mathcal{T}^T = \mathcal{T}^T\mathcal{T} = \boldsymbol{I}$

Karhunen-Loève Transform (KLT)

- For Gaussian-distributed data, the optimal transform is the KLT
- Definition of KLT:

$$\mathcal{T}_{\text{KLT}} = \mathsf{E}^T$$

  where E is the matrix of eigenvectors of the data covariance matrix
- The KLT is also called Principal Component Analysis (PCA) in the context of Machine Learning
- It amounts to align the axis to the direction along which the data have the max variance

Limitations of KLT

- The KLT is data-dependent:
  - Requires knowledge of data statistics or its estimation: this process may be long, costly and inaccurate
  - Transformation matrix changes for each dataset, thus it must be sent with data
  - Computationally expensive
- Solution: use frequency-domain, data independent transforms
  - DFT (Discrete Fourier Transform)
  - DCT (Discrete Cosine Transform)
- These transforms are asymptotically optimal for stationary signals

DFT Transformation Matrix

The DFT transformation matrix is defined as:

$$\mathcal{T}_{\text{DFT}} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & W_N & W_N^2 & \cdots & W_N^{N-1} \\ 1 & W_N^2 & W_N^4 & \cdots & W_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & W_N^{N-1} & W_N^{2(N-1)} & \cdots & W_N^{(N-1)(N-1)} \end{bmatrix}$$

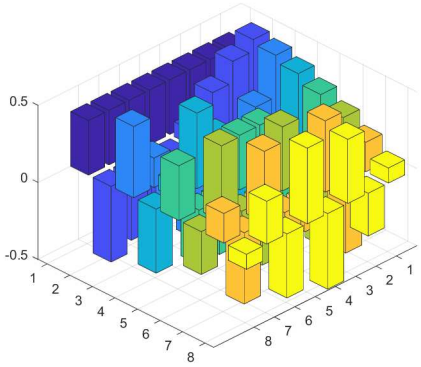
where $W_N = e^{-j2\pi/N}$ is the Nth primitive root of unity.

DCT Transformation Matrix

There exist several variations of the DCT The DCT-II (most common) transformation matrix is defined as:

$$(\mathcal{T}_{\text{DCT}})_{k,n} = \begin{cases} \frac{1}{\sqrt{N}}, & k = 0 \\ \sqrt{\frac{2}{N}} \cos \frac{(2n+1)k\pi}{2N}, & k > 0 \end{cases} \qquad k, n = 0, 1, \ldots, N-1$$

1D Linear Transforms: DCT-8

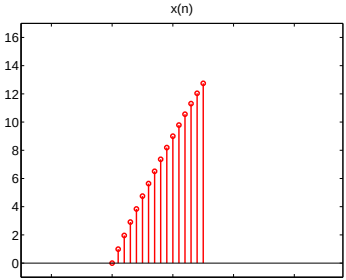


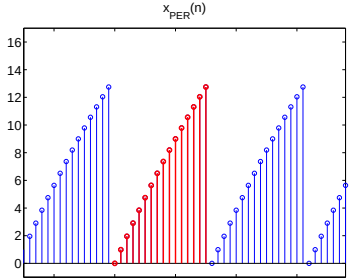
Increasing-frequency basis vectors

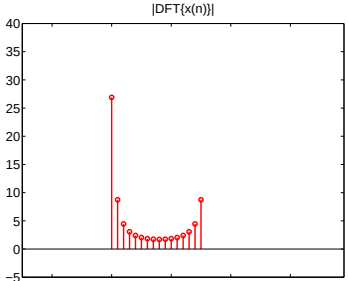
DFT vs DCT

DCT is a better transform than DFT for compression: why?
Remember that the DFT is the discrete-time analogous of the *Fourier Series*.
This means that with the DFT we analyze a periodized version x_{PER} of the original signal x.
This periodized version introduces high-frequency content near the signal boundaries.
Thus, even a smooth signal will have relatively high-frequency content just because of periodization.
A better solution is to *mirror* the signal before periodization: this leads to the DCT

DFT vs DCT


x(n)
16
14
12
10
8
6
4
2
0
−10
0
10
20
30

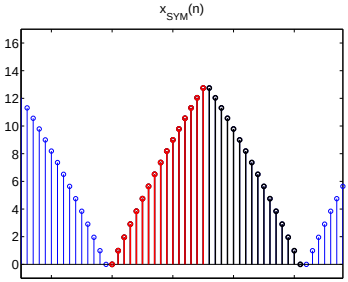

x_PER(n)
16
14
12
10
8
6
4
2
0
−10
0
10
20
30

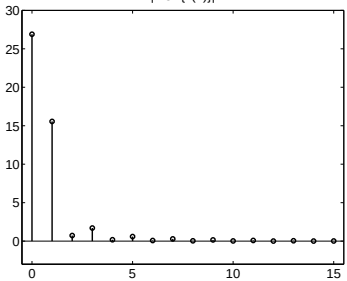

|DFT{x(n)}|
40
35
30
25
20
15
10
5
0
−5
−10
0
10
20
30

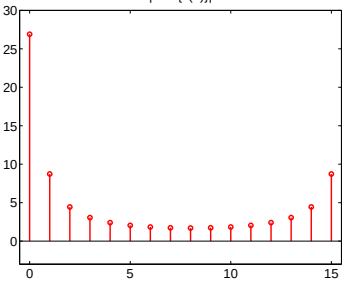
DFT vs DCT

We first create a symmetric mirror of the signal x, then we periodize it, finally we compute the Discrete Fourier Series of the resulting x_{SYM}.
This will mitigate the boundary effect, but at the cost of doubling the period, i.e., doubling the number of coefficients.
However, using some mathematical property of the DFT, we can arrange thing is such a way that the coefficients are symmetrical too, thus only half of them is needed. This process produces the Discrete Cosine Transform of the input signal.

DFT vs DCT


$x_{SYM}(n)$
16
14
12
10
8
6
4
2
0
−10
0
10
20
30


|DCT{x(n)}|
30
25
20
15
10
5
0
0
5
10
15


|DFT{x(n)}|
30
25
20
15
10
5
0
0
5
10
15

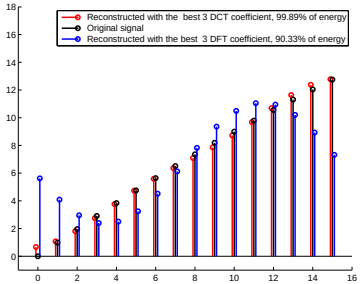
DFT vs DCT

In conclusion, applying the DCT to a signal (a sequence of N real numbers) produces N real coefficients and has a better sparsification property than DFT thanks to the symmetric periodization.
For these reasons, DFT is never used for compression, while DCT is very common.

DFT vs DCT

DCT	
Coeff	% Energy
1	74.61
2	24.98
3	0.05
4	0.30
5	0.00
6	0.04
7	0.00
8	0.01
9	0.00
10	0.00
11	0.00
12	0.00
13	0.00
14	0.00
15	0.00
16	0.00

DFT	
Coeff	% Energy
1	74.61
2	7.86
3	2.04
4	0.97
5	0.60
6	0.43
7	0.35
8	0.31
9	0.30
10	0.31
11	0.35
12	0.43
13	0.60
14	0.97
15	2.04
16	7.86



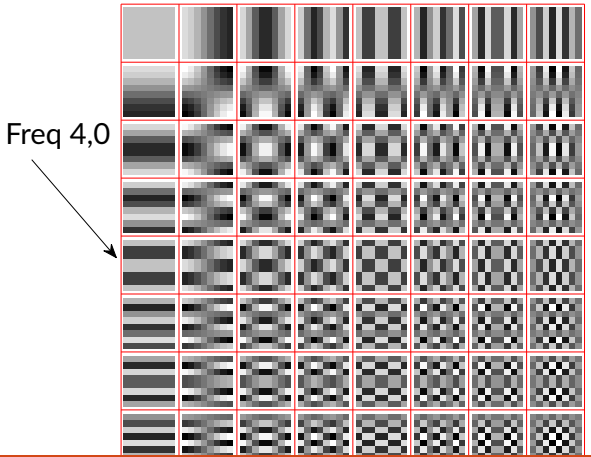
2D Transforms

- 2D transforms are obtained by applying the 1D transform first to rows and then to columns (or vice versa)
- For a data **matrix** X, the 2D transform is:

$$Y = \mathcal{T} X \mathcal{T}^T$$

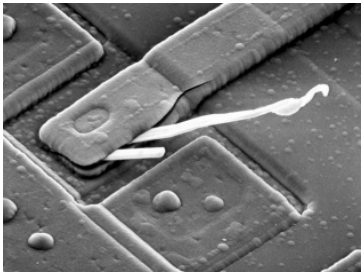
  where $\mathcal{T}$ is the 1D transformation matrix
- This is equivalent to:
  1. Applying the transform to each row of X
  2. Applying the transform to each column of the result

2D Linear Transforms: DCT 8×8

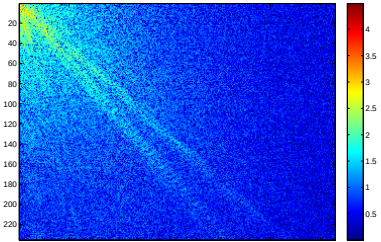


DCT examples

The top left coefficient of the DCT (image on the right) is the “DC component” i.e. the zero-frequency component of the signal. The other coefficients are called AC and represent the direction (angle) and the velocity of spatial variations of the signal (slow variation = low frequency, fast variation –e.g., contours– = high frequencies).
In this example we see two dominant directions of the contours. This corresponds to the two “traces” in the DCT image



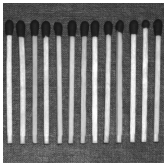
Image

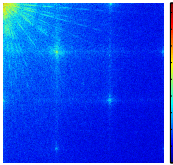


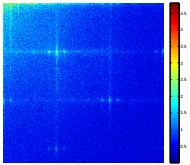
DCT coeff's logarithm

DCT examples

In the left image, the contours have a lot of different directions: we see many “traces” in the DCT. The two “impulses” in the DCT image represent the periodic component of the background fabric.
In the right image the diagonal traces disappear, while the impulses are still there because the fabric is the same

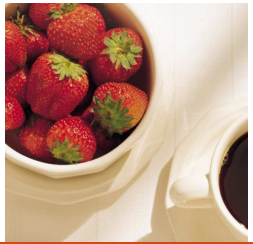


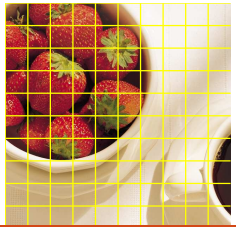


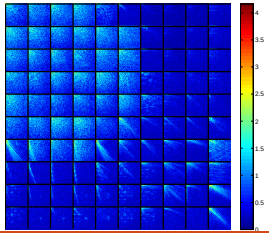


Block-based DCT

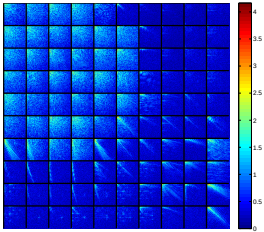
- A large-size non-stationary image is more conveniently represented by dividing it into small blocks.
- Blocks from detailed areas have many non-negligible coefficients (strawberries)
- Block with strong contours have only directional traces (e.g, bottom right blocks)
- “Simple” blocks (e.g., top-right) have only very few non-negligible coefficients
- According to the image, one could find a block size that makes the signal the sparsest
- The best approach is to use variable block size (not used in JPEG)

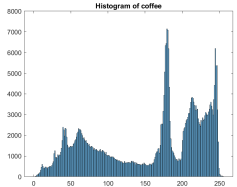


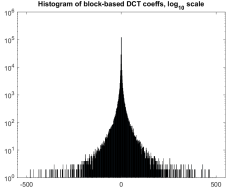




DCT examples







DCT: sparsity

- DCT allows to **sparsify** images
- This means that we pass from an image where all the samples are equally important to a set of coefficients where many are negligible (i.e., small-valued) and just a few are large
- This enables compression since we can give many coding bits to the few large coefficients and much less bits to the large majority of small coefficients, without degrading much the image quality
- Next labs allow to better understand this issue, showing that, after DCT, most of the image content is recoverable even by neglecting a large part of transformed coefficients

Resource allocation for DCT coefficients

Once we apply the DCT to an image, how to decide the quantization step of each coefficient?

- Huang-Schulteiss formula, or Greedy Algorithm
  - See next slide for an example
  - Very simple solution
  - Very common in practice (e.g., quantization matrix in JPEG)

Summary

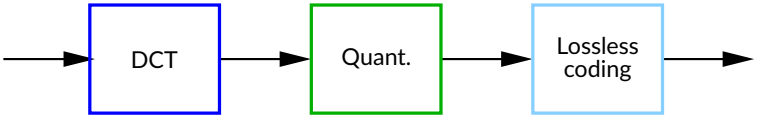
1. **Block coding**: take a joint decision on the quantization rate of a block of data, looking for better RD (rate-distortion) performance than the case of sample-by-sample quantization
2. **Huang-Schulteiss formula**: gives the best coding rate for each component of a random vector $\boldsymbol{X}$
3. **Coding gain**: shows that total distortion is proportional to the *geometric mean* of the component variances, σ^2_{GM}
4. **Orthogonal transforms**: allow to modify (reduce) σ^2_{GM}
5. **Practical rate allocation**: modified H.S., greedy allocation, fixed allocation can all be used in practical cases; first 2 are equivalent, third one is simpler and often used
6. **Frequency transforms**: convenient special cases of orthogonal transforms

Outline

JPEG Standard

- Image compression standard based on DCT
- Specified in 1991, adopted in 1992
- It describes *only the decoder*: this insures **interoperability** and **implementation competition**
- It can work on multi-component (color) images, but *we will describe its operation on one component*, the other being identical
  - One-component images are assumed to be gray-scale
  - Three-component images are assumed to be YCbCr, unless specified otherwise in a marker

JPEG Standard: Scheme



- The image is decomposed into 8×8 pixels blocks
- The value 128 is subtracted from the image
- Blocks are (almost) independently encoded

JPEG Standard: Blocks

Example of 8x8 block

173	171	171	143	109	100	91	96
171	169	150	137	112	101	94	96
184	158	139	120	110	107	94	100
170	156	134	119	117	104	98	99
157	147	125	127	103	109	90	98
149	146	132	120	113	107	101	93
147	141	119	119	111	101	100	92
160	122	117	116	115	116	102	95

JPEG Standard: Transform

- DCT is performed on 8×8 blocks
- Small blocks ensures stationarity
- Large blocks better exploit correlation
- Size chosen after experiences
- Coefficients DCT : impact on SVH

JPEG Standard: Transform

Coefficients DCT of the block

985.3	186.2	34.1	11.6	7.3	1.6	4.9	-8.2
40.3	47.8	5.7	-26.0	-5.3	-3.5	4.0	-1.0
6.3	4.0	-9.3	-6.7	-1.2	8.1	3.4	4.1
-0.0	4.9	-13.3	-20.8	-10.4	-1.0	-4.5	-5.1
2.1	-1.3	-1.6	0.6	3.6	3.3	8.1	-1.7
1.3	3.7	2.4	-2.7	-2.2	-3.0	-4.1	7.8
5.1	0.4	3.1	4.8	-1.4	2.5	9.8	5.3
-5.6	1.6	4.4	0.1	3.3	2.3	4.3	-8.4

JPEG Standard: Transform

Standard deviation of DCT coefficients for natural images

396.64	100.99	49.26	31.15	19.74	14.57	8.76	7.33
100.23	55.78	37.40	24.77	16.44	11.70	8.44	6.25
49.42	36.39	28.01	20.40	14.64	10.46	7.64	5.88
30.82	24.05	19.73	15.47	11.99	8.88	6.83	5.45
21.09	16.79	14.79	11.54	9.19	7.30	5.90	4.68
15.32	11.91	10.31	8.71	7.15	5.78	4.61	3.91
11.22	8.58	7.66	6.78	5.69	4.64	3.82	3.24
8.21	6.65	5.93	5.52	4.45	3.75	3.15	2.80

JPEG Standard: Quantization

- Uniform quantization
- $\widetilde{c}_{i,j} = \text{round}\left(\frac{c_{i,j}}{q_{i,j}}\right)$
- The quantization q table defines the RD trade-off, but it is not defined by the standard
- Thus q must be encoded

JPEG Standard: Quantization

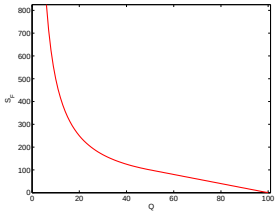
Quantization table example

$$q^* = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 81 & 61 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 111 & 90 \\ 49 & 63 & 78 & 87 & 101 & 121 & 120 & 100 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

JPEG Standard:Quality factor

- It is a non-normative tool that eases the selection of a quantization matrix
- The quality factor Q is between 1 and 100
- It defines a scaling factor S_F for the quantization matrix

$$S_F = \begin{cases} \frac{5000}{Q} & 1 \leq Q \leq 50 \\ 200 - 2Q & 50 < Q \leq 99 \\ 1 & Q = 100 \end{cases}$$



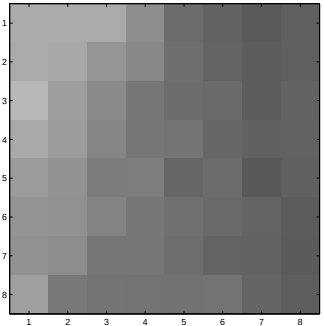
$$q \leftarrow \frac{S_F}{100} q^*$$

JPEG Standard: Quantization

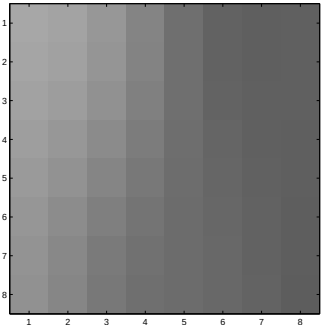
Quantized coefficients

61	16	3	0	0	0	0	0
3	3	0	-1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

JPEG Standard: Example

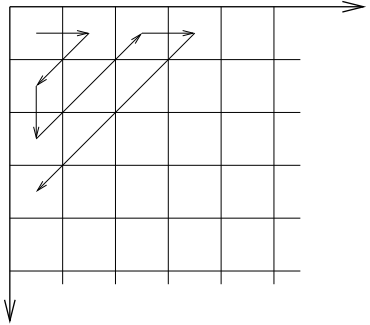


Original



$\mathrm{DCT} \rightarrow Q \rightarrow Q^{-1} \rightarrow \mathrm{DCT}^{-1}$

JPEG Standard: Entropy coding



- Zig-zag scan

JPEG Standard: Entropy coding

- DC Coefficient : prediction + Huffman
- AC Coefficients : “run-length” + Huffman

coeff$\neq 0$	# of 0's	coeff$\neq 0$	# of 0's	⋯	EOB	⋯

JPEG Standard: Entropy coding

Encoded quantized coefficients of the n-th block

61	16	3	0	0	0	0	0
3	3	0	-1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

61-DC_{n-1}	0,16	0,3	1,3	0,3	7,-1	EOB

JPEG: Entropy coding of DC coeffs

- DC Coeff's difference are represented by the category/amplitude couple
- There are 12 categories, named $0, \ldots, 11$, coded with a *pseudo-Huffman* code
- The category k of the DC prediction error DC_P is defined as: $\lceil \log_2 (|\mathrm{DC}_P| + 1) \rceil$
- In other words, category k contains 2^k values (or amplitudes): $\{\pm 2^{k-1}, \ldots, \pm 2^k - 1\}$; each amplitude is coded on k bits, using 2's complements for negative values
- The DC prediction error is then encoded by writing first the bits of the category and then those of the value (or amplitude)

JPEG: Entropy coding of DC coeffs

Here we show the tables for DC categories and for encoding an amplitude given the category:

Category	Code
0	00
1	010
2	011
3	100
4	101
5	110
6	1110
7	11110
8	111110
9	1111110
10	11111110
11	111111110

Category	Amplitude	Code
0	0	-
1	+1	1
	-1	0
2	+2	10
	+3	11
	-2	01
	-3	00
3	+4	100
	+5	101
	…	…
	-7	000
…	…	…
11	1024	1 000 0000 000
	…	…
	-2047	0 000 0000 000

- For example, if the prediction error is $\mathrm{DC}_P = \mathrm{DC}_n - \mathrm{DC}_{n-1} = -5$, we first find the category $k = \lceil \log_2 (|\mathrm{DC}_P| + 1) \rceil = 3$
- Now we write the code of category: `100`; and 5 in binary: `101`, but we complement each bit because the value is negative: finally we encode: `100010`
- If we were to encode $+5$, we would have `100101`

JPEG: Entropy coding of AC coeffs

- AC coeffs are represented by run-length, category, amplitude.
- Categories and run-lengths are coded together using a table (again, a pseudo-Huffman coding)
- The table is not standardized and is specified in the file header
  - Special symbol 1: (15,0) means “at least 15 zeros before next non-zero coefficient” (Zero-Run)
  - Special symbol 2: (0,0) means “end of block” (EOB)
- The amplitude is encoded exactly as as in the DC case

JPEG: Entropy coding of AC coeffs

R/C	Code	R/C	Code	R/C	Code
0/0 (EOB)	1010				
0/1	00	2/1	11100	4/1	111011
0/2	01	2/2	11111001	4/2	1111111000
0/3	100	2/3	1111110111	4/3	1111111110010110
0/4	1011	2/4	111111110100	4/4	1111111110010111
0/5	11010	2/5	1111111110001001	4/5	1111111110011000
0/6	1111000	2/6	1111111110001010	4/6	1111111110011001
0/7	11111000	2/7	1111111110001011	4/7	1111111110011010
0/8	1111110110	2/8	1111111110001100	4/8	1111111110011011
0/9	1111111110000010	2/9	1111111110001101	4/9	1111111110011100
0/10	1111111110000011	2/10	1111111110001110	4/10	1111111110011101
1/1	1100	3/1	111010	5/1	1111010
1/2	11011	3/2	111110111	5/2	11111110111
1/3	1111001	3/3	111111110101	5/3	1111111110011110
1/4	111110110	3/4	1111111110001111	5/4	1111111110011111
1/5	11111110110	3/5	1111111110010000	5/5	1111111110100000
1/6	1111111110000100	3/6	1111111110010001	5/6	1111111110100001
1/7	1111111110000101	3/7	1111111110010010	5/7	1111111110100010
1/8	1111111110000110	3/8	1111111110010011	5/8	1111111110100011
1/9	1111111110000111	3/9	1111111110010100	5/9	1111111110100100
1/10	1111111110001000	3/10	111111110010101	5/10	1111111110100101

JPEG: Entropy coding of AC coeffs

R/C	Code	R/C	Code	R/C	Code
6/1	1111011	8/1	111111000	A/1	111111010
6/2	111111110110	8/2	1111111111000000	A/2	1111111111000111
6/3	1111111110100110	8/3	1111111110110110	A/3	1111111111001000
6/4	1111111110100111	8/4	1111111110110111	A/4	1111111111001001
6/5	1111111110101000	8/5	1111111110111000	A/5	1111111111001010
6/6	1111111110101001	8/6	1111111110111001	A/6	1111111111001011
6/7	1111111110101010	8/7	1111111110111010	A/7	1111111111001100
6/8	1111111110101011	8/8	1111111110111011	A/8	1111111111001101
6/9	1111111110101100	8/9	1111111110111100	A/9	1111111111001110
6/10	1111111110101101	8/10	1111111110111101	A/10	1111111111001111
7/1	11111010	9/1	111111001	B/1	1111111001
7/2	111111110111	9/2	1111111110111110	B/2	1111111111010000
7/3	1111111110101110	9/3	1111111110111111	B/3	1111111111010001
7/4	1111111110101111	9/4	1111111111000000	B/4	1111111111010010
7/5	1111111110110000	9/5	1111111111000001	B/5	1111111111010011
7/6	1111111110110001	9/6	1111111111000010	B/6	1111111111010100
7/7	1111111110110010	9/7	1111111111000011	B/7	1111111111010101
7/8	1111111110110011	9/8	1111111111000100	B/8	1111111111010110
7/9	1111111110110100	9/9	1111111111000101	B/9	1111111111010111
7/10	1111111110110101	9/10	1111111111000110	B/10	1111111111011000

JPEG: Entropy coding of AC coeffs

R/C	Code	R/C	Code
C/1	1111111010	E/1	1111111111101011
C/2	1111111111011001	E/2	1111111111101100
C/3	1111111111011010	E/3	1111111111101101
C/4	1111111111011011	E/4	1111111111101110
C/5	1111111111011100	E/5	1111111111101111
C/6	1111111111011101	E/6	1111111111110000
C/7	1111111111011110	E/7	1111111111110001
C/8	1111111111011111	E/8	1111111111110010
C/9	1111111111100000	E/9	1111111111110011
C/10	1111111111100001	E/10	1111111111110100
		F/0 (ZR)	11111111001
D/1	11111111000	F/1	1111111111110101
D/2	1111111111100010	F/2	1111111111110110
D/3	1111111111100011	F/3	1111111111110111
D/4	1111111111100100	F/4	1111111111111000
D/5	1111111111100101	F/5	1111111111111001
D/6	1111111111100110	F/6	1111111111111010
D/7	1111111111100111	F/7	1111111111111011
D/8	1111111111101000	F/8	1111111111111100
D/9	1111111111101001	F/9	1111111111111101
D/10	1111111111101010	F/10	1111111111111110

JPEG Standard: Example of entropy coding

Let us suppose that $\mathrm{DC}_{n-1} = 66$. We have to encode:

61-DC_{n-1}	0,16	0,3	1,3	0,3	7,-1	EOB

We have:

$\mathrm{DC}_P = -5$	Category=3 → 100	Value=-5 → 010	100010
$(R, V) = (0, 16)$	Run, Category = (0,5) → 11010	Value=16 → 10000	1101010000
$(R, V) = (0, 3)$	Run, Category = (0,2) → 01	Value=3 → 11	0111
$(R, V) = (1, 3)$	Run, Category = (1,2) → 11011	Value=3 → 11	1101111
$(R, V) = (0, 3)$	Run, Category = (0,2) → 01	Value=3 → 11	0111
$(R, V) = (7, -1)$	Run, Category = (7,1) → 11111010	Value=-1 → 0	111110100
EOB			1010

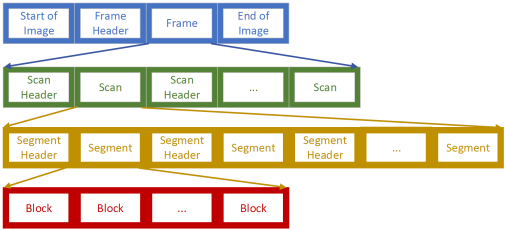
The block is encoded as: `100010 1101010000 01 11 11011 11 01 11 11111010 0 1010`

Note that this is a prefix-code, thus no “separator” is needed.

We use 44 bits per 64 pixels, averaging ≈ 0.69 bpp

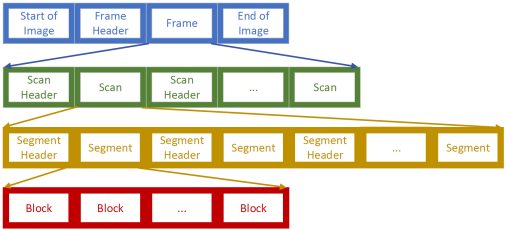
Note also that it is difficult to tell in advance the number of bits used to encode a block as a function of the quality factor or of the quantization table: thus *rate control* is not a feature of JPEG

Frame Building



- An image is stored into a `frame`
- The `frame header` tells the image size, the components (single component, RGB, YCbCr, ...), the Digitization format (4:2:0,...)
- Tthe `frame` contains one `scan` per component (baseline JPEG)
- The `scan header` identifies the component and specifies the quantization tables: typically, one for the luminance and another for the chrominance

Frame Building



- A scan is made up of `segments`, optionally separated by `restart` markers used for parallel decoding
- The `segment header` defines the Huffman tables. Typically four different tables can be used, differentiating DC vs. AC and Y vs. CbCr
- A segment is made up of `blocks`. A block is the encoding of 8×8 pixels and does not have headers

JFIF (JPEG File Interchange Format)

- Standard format for including metadata in JPEG files
- Primary purpose: Ensure interoperability between JPEG-using devices and software
- Contains basic information:
  - JFIF version
  - Density units (pixels per inch, pixels per centimeter)
  - X and Y density (resolution)
  - Thumbnail (optional)
- Does not include detailed camera or shooting settings information
- Widely supported and used in web images

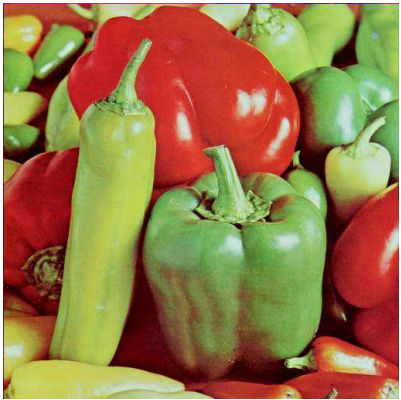
Exif (Exchangeable Image File Format)

- Rich metadata format designed specifically for digital cameras
- Developed by Japan Electronic Industries Development Association (JEIDA)
- Can contain a wide range of information:
  - Camera make and model
  - Date and time of shot
  - Camera settings (aperture, shutter speed, ISO, etc.)
  - GPS information (if available)
  - Thumbnail image
  - Copyright information
- Widely supported by digital cameras and photo editing software
- Can include manufacturer-specific proprietary data
- Preferred by photographers for its detailed metadata

JPEG Standard: Coding example

Original image

JPEG Standard: Coding example



Rate=1.02 bpp PSNR=33.92 dB

JPEG Standard: Coding example

Rate=0.75 bpp PSNR=33.45 dB

JPEG Standard: Coding example

Rate=0.50 bpp PSNR=32.70 dB

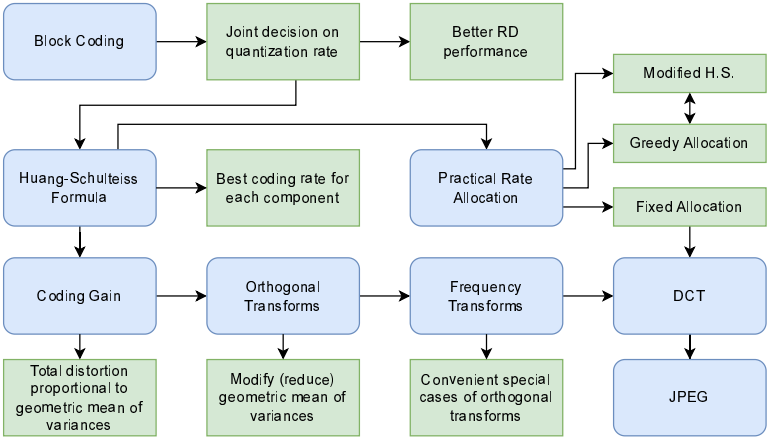
JPEG Standard: Coding example

Rate=0.31 bpp PSNR=31.31 dB

JPEG Standard: Coding example

Rate=0.21 bpp PSNR=29.50 dB

Transform coding: conceptual map



JPEG: conceptual map